Střední průmyslová škola, Ústí nad Labem, Resslova 5

Playstorm

Dokumentace k ročníkové práci

Autor: Martin Černý

Třída: 4ITA

Vedoucí práce: Bc. Tomáš Ulrich

2025/2026

Prohlášení

Prohlašuji, že jsem ročníkovou práci na téma „Playstorm“ vypracoval samostatně s použitím uvedené literatury a pramenů.

V Ústí nad Labem dne 30.12.2025

…….……..

Poděkování

Chtěl bych poděkovat Bc. Tomáši Ulrichovi za vedení mé ročníkové práce, cenné rady a odborný dohled.

Anotace

Tento dokument slouží jako kompletní technická dokumentace k ročníkové práci s názvem „Playstorm“, která se zabývá vývojem webové platformy pro synchronizované streamování videa. Text je systematicky rozdělen do několika klíčových částí, které čtenáře provádějí celým procesem od teoretického základu až po praktickou realizaci. Úvodní pasáže dokumentu jsou věnovány rešerši existujících služeb, konkrétně platformám Netflix a Watch2Gether, které byly hlavní funkční i designovou inspirací pro tento projekt. Navazující teoretický blok podrobně rozebírá použitý technologický stack, kde jsou představeny nástroje pro frontend (React, Vite), backend (NestJS, Go) a správa databázové infrastruktury pomocí PostgreSQL a Dockeru. Praktická část dokumentace se zaměřuje na samotný návrh systému, včetně popisu databázového schématu (ERD) a architektury rozhraní API. Stěžejní částí textu je kapitola o produktizaci, která na konkrétních ukázkách zdrojového kódu vysvětluje vnitřní logiku aplikace, vyhledávání médií a především mechanismus real-time synchronizace uživatelů v rámci funkce WatchParty. Dokumentaci uzavírá uživatelský popis ovládání aplikace, shrnutí dosažených výsledků v závěru a seznam použitých informačních zdrojů a obrazových příloh.

Klíčová slova

Kód, Media-Server, ERD, Databáze, Streamování, WatchParty, Real-time synchronizace, NestJS, React, Docker.

Obsah

Obsah

Úvod ... 8
1 Teoretická část ... 9
1.1 Rešerše ... 9
1.1.1 Netflix ... 9
1.1.2 Watch2Gether ... 9
1.2 Technologie ... 10
1.2.1 React & Vite ... 10
1.2.2 Typescript ... 10
1.2.3 Tailwind CSS ... 10
1.2.4 Socket.io ... 10
1.2.5 Vidstack ... 10
1.2.6 Zustand ... 10
1.2.7 NestJS ... 10
1.2.8 Prisma ... 10
1.2.9 Postgresql ... 11
1.2.10 Argon 2 & JWT ... 11
1.2.11 FFmpeg ... 11
1.2.12 Python ... 11
1.2.13 Docker ... 11
2 Praktická část ... 12
2.1 Návrhy ... 12
2.1.1 Návrh Databáze ... 12
2.1.2 Návrh backend routing a API ... 13
2.2 Produktizace ... 14
2.2.1 Metoda getVideo() ... 14
2.2.2 Vyhledávání epizody na media serveru ... 15
2.2.3 Ovládání přehrávání ve WatchParty ... 16
2.2.4 Modal Provider ... 17
2.2.5 Dynamické přidávání nastavení ... 18
2.2.6 Zabezpečené Přihlašování ... 19
2.2.7 Synchronizace eventů WatchParty ... 20
2.2.8 Globální správa stavu přehrávače ... 21
2.3 Popis pro uživatele ... 22

Závěr 23
Seznam použitých zdrojů 24
Seznam obrázků 25
Seznam kódů 25

Úvod

Hlavním cílem ročníkové práce je vytvořit webovou aplikaci streamovací platformy „PlayStorm“, která uživatelům umožní pohodlné sledování multimediálního obsahu, jako jsou filmy a seriály. Klíčovou přidanou hodnotou projektu je implementace funkce „WatchParty“, která má za cíl zprostředkovat sdílený zážitek ze sledování v reálném čase. Tato funkce by měla umožnit skupině uživatelů synchronizovaně ovládat přehrávání, což v praxi znamená, že akce jako pauza nebo přetáčení provedené jedním účastníkem se okamžitě projeví u všech ostatních.

K výběru tohoto tématu mě vedl hluboký zájem o systémový design a architekturu velkých streamovacích služeb, jako jsou Netflix nebo HBO. Fascinuje mě technologické pozadí těchto platforem, zejména způsob, jakým efektivně pracují s videem ve vysokém rozlišení a jak zajišťují plynulý uživatelský zážitek. Tvorba vlastní platformy je pro mě ideální příležitostí, jak si v praxi vyzkoušet moderní technologie, které stojí za současným webovým streamováním.

Od práce očekávám především prohloubení svých znalostí v oblasti full-stack vývoje, konkrétně při propojování frontendové části postavené na knihovně React s robustním backendem v NestJS. Velkou výzvou, na kterou se v projektu zaměřím, je vyřešení technických aspektů real-time synchronizace uživatelů pomocí webových soketů. Mým záměrem na začátku projektu je navrhnout a vytvořit plynulou, bezpečnou a funkční aplikaci s intuitivním ovládáním, která lidem nabídne kvalitní zážitek ze společného sledování médií.

1 Teoretická část

1.1 Rešerše

1.1.1 Netflix

Netflix je globální streamovací platforma zaměřená na online přehrávání filmů a seriálů s důrazem na jednoduché ovládání a personalizaci obsahu. Inspirací pro aplikaci PlayStorm je především přehledné uživatelské rozhraní, které klade obsah do popředí, a princip doporučování na základě chování uživatele. Důležitým prvkem je také stabilní a plynulé streamování, které zajišťuje kvalitní uživatelský zážitek. (1)

Obrázek 1 - Netflix Logo

1.1.2 Watch2Gether

Online služba umožňující více uživatelům sledovat video obsah společně v reálném čase. Hlavní inspirací pro aplikaci PlayStorm je koncept synchronizovaného přehrávání a sdíleného zážitku na dálku. Tato služba ukazuje, jak lze propojit přehrávání videa s interakcí mezi uživateli, což je klíčový princip celé aplikace PlayStorm. (2)

Obrázek 2 - Watch2Gether Logo

1.2 Technologie

1.2.1 React & Vite

React je moderní knihovna v JavaScriptu, která se používá pro tvorbu uživatelských rozhraní a je postavená na komponentách, díky čemuž je kód přehledný a dobře se udržuje. Pracuje s virtuálním DOM, což umožňuje rychlé a efektivní aktualizace stránky. Vite je nástroj určený pro vývoj a sestavení aplikací, který výrazně zrychluje práci díky okamžitému spuštění projektu a rychlému obnovení změn bez nutnosti zdlouhavého buildování projektu. (3) (4)

1.2.2 Typescript

Typescript je nadstavba jazyka Javascript která nabízí statické typování a další nástroje pro kontrolu kódu a zamezení nekonzistenci typů při práci s daty. (5)

1.2.3 Tailwind CSS

Tailwind CSS je moderní CSS Framework který umožnuje úpravu a tvorbu CSS stylů přímo v html kódu bez nutnosti tvorby stylových souborů CSS (6)

1.2.4 Socket.io

Socket.io je knihovna využívající webové sokety pro komunikaci v reálném čase mezi klientem a serverem. Umožňuje práci s daty mezi klientem a serverem bez nutnosti obnovování nebo znovu načítání stránky a změna dat se tedy projeví hned jakmile se provede. (7)

1.2.5 Vidstack

Vidstack je moderní knihovna pro práci s videem na webu. Slouží jako alternativa k běžným HTML5 přehrávačům. Poskytuje již vytvořené komponenty a API pro snadnou integraci videa/streamů nebo vlastních ovládacích prvků pro aplikaci. Jeho hlavním benefitem je absolutní volnost stylizace přehrávače a nastavení argumentů pro přijímaný obsah. (8)

1.2.6 Zustand

Zustand je knihovna pro správů stavů v aplikaci. Umožňuje ukládat a spravovat globální stavy bez složité konfigurace. Díky jednoduchému API je práce se stavem přehledná a efektivní, což z ní dělá vhodné řešení jak pro menší projekty (9)

1.2.7 NestJS

NestJS je moderní backendový framework postavený na Node.js, který využívá TypeScript a je inspirovaný architekturou Angularu. Je založený na modulárním principu, používá dependency injection a jasně odděluje jednotlivé části aplikace, což výrazně zvyšuje přehlednost a udržovatelnost kódu. NestJS se často používá pro tvorbu REST API nebo real-time serverů, je dobře škálovatelný a vhodný jak pro menší projekty, tak i pro rozsáhlé backendové aplikace. (10)

1.2.8 Prisma

Prisma je moderní ORM nástroj pro práci s databázemi v JavaScriptu a TypeScriptu, který výrazně zjednodušuje komunikaci mezi aplikací a databází. Umožňuje definovat databázový model pomocí přehledného schématu, ze kterého automaticky generuje typově bezpečný klient pro práci s daty. Díky tomu je kód čitelnější, méně náchylný k chybám a dobře se používá například v kombinaci s NestJS nebo Express, nejčastěji s databázemi jako PostgreSQL nebo MySQL. (11)

1.2.9 Postgresql

PostgreSQL je výkonný open-source relační databázový systém, který se používá pro ukládání a správu dat v moderních aplikacích. Nabízí vysokou spolehlivost, bezpečnost a podporu pokročilých funkcí, jako jsou transakce, indexy nebo práce s relačními daty. Díky dobré škálovatelnosti a kompatibilitě s nástroji jako Prisma nebo backendovými frameworky typu NestJS je PostgreSQL častou volbou pro menší i rozsáhlé projekty. (12)

1.2.10 Argon 2 & JWT

Argon2 a JWT jsou technologie používané především v oblasti autentizace a zabezpečení aplikací. Argon2 je moderní a bezpečný algoritmus pro hashování hesel, který chrání uživatelská data tím, že hesla ukládá v nevratné podobě a je odolný vůči útokům hrubou silou. JWT (JSON Web Token) slouží k bezpečnému přenosu informací o uživateli mezi klientem a serverem, nejčastěji ve formě přihlašovacího tokenu. Společně se tyto technologie používají k vytvoření bezpečného přihlašovacího systému v moderních webových aplikacích. (13)

1.2.11 FFmpeg

FFmpeg je výkonný open-source nástroj určený pro práci s multimediálním obsahem, především pro zpracování videa a audia. Umožňuje převod formátů, úpravu kvality, extrakci zvuku, práci se streamy nebo základní editaci médií pomocí příkazové řádky. FFmpeg se často používá na serverech a v automatizovaných systémech, kde je potřeba efektivně zpracovávat multimediální soubory. (14)

1.2.12 Python

Python je vysokoúrovňový programovací jazyk známý svou jednoduchou a čitelnou syntaxí, díky které je vhodný jak pro začátečníky, tak pro pokročilé vývojáře. Používá se v širokém spektru oblastí, jako je backendový vývoj, automatizace, práce s daty nebo umělá inteligence. Python má rozsáhlý ekosystém knihoven a frameworků, což umožňuje rychlý vývoj aplikací a snadné řešení komplexních úloh. (15)

1.2.13 Docker

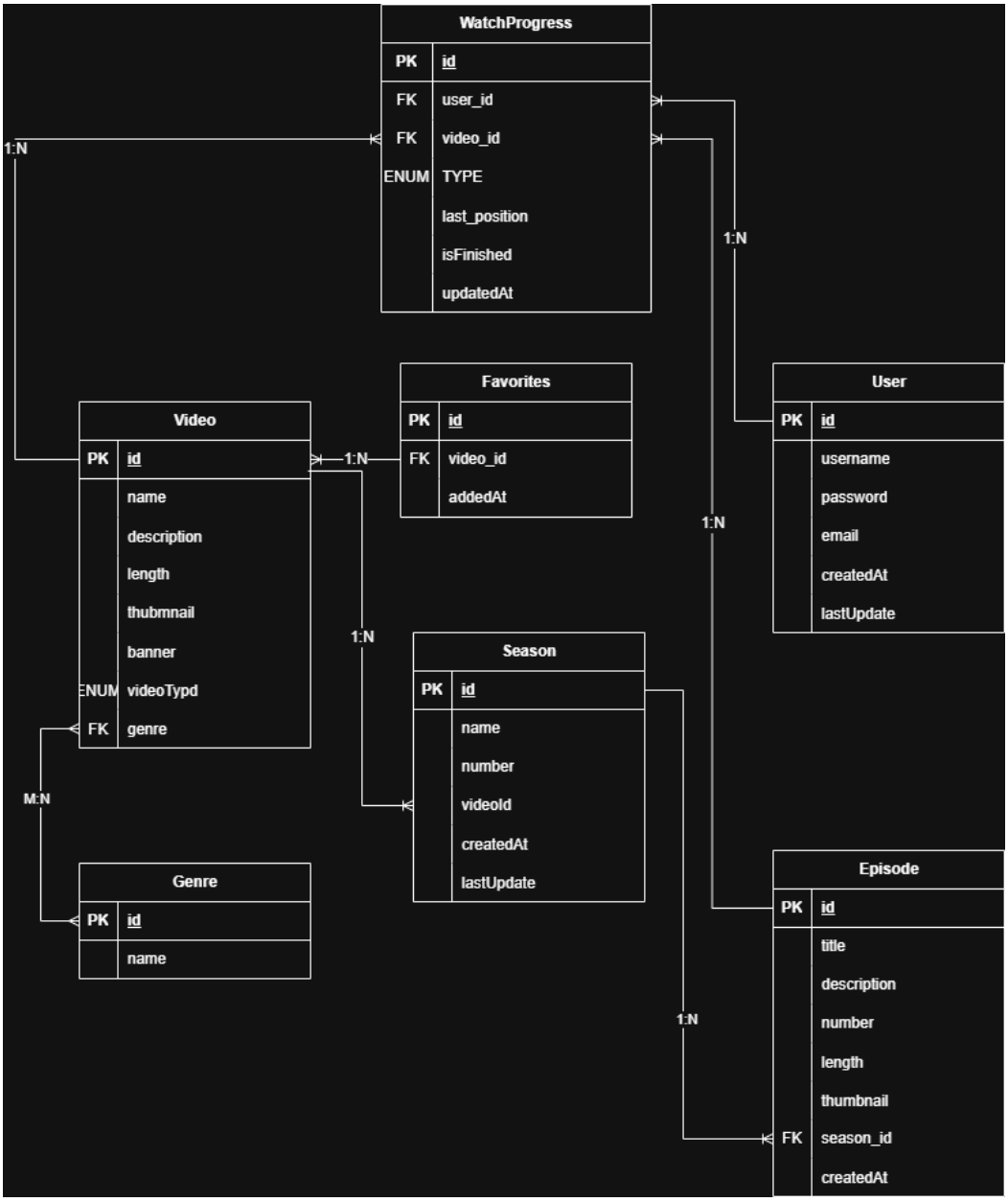
Docker je nástroj pro kontejnerizaci aplikací, který umožňuje zabalit aplikaci spolu se všemi jejími závislostmi do jednoho izolovaného prostředí zvaného kontejner. Díky tomu aplikace běží konzistentně na různých systémech bez ohledu na prostředí, ve kterém je spuštěna. Docker se běžně používá pro nasazování aplikací, zjednodušení správy serverů a vývoj moderních backendových i full-stack projektů, kde je důležitá přenositelnost a škálovatelnost. (16)

2 Praktická část.

2.1 Návrhy

2.1.1 Návrh Databáze

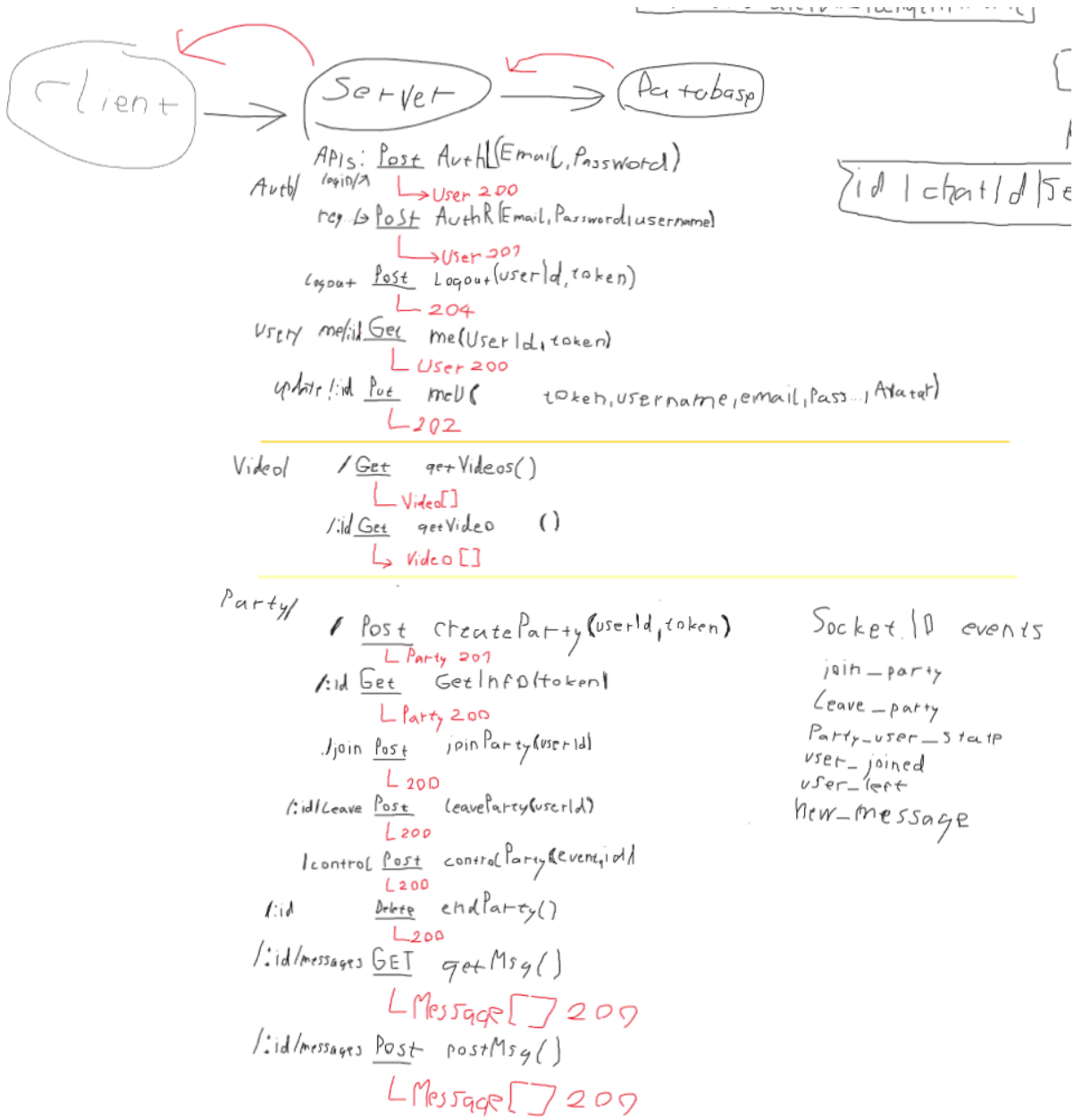
Vytvořil jsem si ERD návrh databáze aby pro mě byla tvorba schéma databáze v Prismě jednodušší a abych mohl lépe pracovat s případnými opravami a úpravami kterých bylo během tvoření projektu nemálo. Do rozložení tohoto návrhu jsem se inspiroval z různých schémat větších streamovacích platforem jako Netflix, HBO apd. Databáze odkazuje spoustu 1:N vztahů což je u těchto streamovacích služeb celkem častý jev. tento návrh mi pomohl při hledání problémů během vývoje backendu a při tvorbě návrhu pro celou restful API. V databázi jsou zvlášť řešené filmy a zvlášť seriály.



Obrázek 3 - ERD Diagram

2.1.2 Návrh backend routing a API

Zde je obrázek původního návrhu celého backendu. Je to původní návrh a během tvorby aplikace musel projít pár změnami. dost mi ale pomohl naučit se vytvářet HLD (High Level Design) a LLD (Low-Level Design) který můžeme vidět na obrázku níže. jedná se o nástřel základní architektury celého backendu a cest které na něm běží. jsou zde zvlášť tři základní cesty Autorizace, Video, a WatchParty plugin. najdeme v návrhu jak metody které používáme pro volání cesty (GET, POST, PUT, DELETE, UPDATE) tak i jejich názvy.



Obrázek 4 - Navrh Backendu a API

2.2 Produktizace

2.2.1 Metoda getVideo()

Tato metoda se nachází na backendu na URL …/video/stream/:id. Tato metoda přijímá jako parametr UUID. Toto UUID se rovná některému požadovanému filmu/epizodě seriálu Tato metoda se první pokusí najít zda poskytnuté ID patří některé epizodě seriálu. Pokud ano vrátí jeho název a typ. pokud ne podívá se zda se jedná o film a případně vrátí název filmu. Tato metoda slouží jako rychlá autorizace zda film/seriál má záznam a tudíž se nachází na media serveru a lze ho načíst a spustit.

```
1.      async getVideos() {
2.          this.logger.log('Fetching all videos');
3.          // ... (keeping getVideos logic)
4.          try{
5.              const videos = await this.prisma.video.findMany({
6.                  take: 40,
7.                  select: {
8.                      id: true,
9.                      name: true,
10.                     thumbnail: true,
11.                     videotype: true,
12.                     genre_id: true,
13.                     watchProgress: {
14.                         select: {
15.                             last_position: true,
16.                             isFinished: true
17.                         }
18.                     }
19.                 }
20.             })
21.
22.             return videos.map((video) => ({
23.                 ...video,
24.                 watchProgress: video.watchProgress.map((progress) => ({
25.                     ...progress,
26.                     last_position: Number(progress.last_position),
27.                 })),
28.             }));
29.         }
30.         catch(err){
31.             this.logger.error(`Failed to fetch videos: ${err.message}`);
32.             throw new InternalServerErrorException(err)
33.         }
34.     }
35.
```

Kód 1 - getVideo()

2.2.2 Vyhledávání epizody na media serveru

Tato metoda streamVideo je v podstatě "mozkem" odchozího streamu na backendu. Stará se o to, aby aplikace věděla, odkud přesně ze složky saved_videos má tahat data pro přehrávač.. Nejdřív si kód složí cestu k videu podle jeho unikátního ID a pro jistotu si checkne, jestli ta složka na disku vůbec je. Pokud ji nenajde, vyhodí chybu a končí. Je tam taky vychytávka pro soubory typu .m3u8 – když si přehrávač řekne o univerzální index.m3u8, metoda zkusí zjistit, jestli neexistuje spíš master.m3u8, který obsahuje víc informací o kvalitě streamu, a kdyžtak ho automaticky podstrčí. Jakmile má cesta k souboru zelenou, vytvořím pomocí createReadStream proud dat, který se nečte celý najednou do paměti, ale posílá se uživateli po kouscích, což je pro streamování videa klíčové. Nakonec ještě nastavím správný "Content-Type" – buď pro playlist (m3u8), nebo pro samotné kousky videa (.ts), aby prohlížeč věděl, co mu to vlastně posílám, a celé to zabalím do objektu StreamableFile. Je to v podstatě taková automatizovaná výhybka, která hlídá, aby data tekla hladce a do správného cíle.

```
1. async streamVideo(id: string, filename: string): Promise<StreamableFile> {
2.     this.logger.log(`Streaming video: ${id} file: ${filename}`);
3.     try {
4.         const baseDir = path.join(process.cwd(), 'saved_videos', id);
5.         try {
6.             await access(baseDir, constants.F_OK);
7.         } catch {
8.             throw new NotFoundException(`Stream directory for ID ${id} not found.`);
9.         }
10.        let actualFilename = filename;
11.        if (filename === 'index.m3u8') {
12.            try {
13.                await access(path.join(baseDir, 'master.m3u8'), constants.F_OK);
14.                actualFilename = 'master.m3u8';
15.            } catch {
16.                actualFilename = 'index.m3u8';
17.            }
18.        }
19.
20.        const requestedFile = path.join(baseDir, actualFilename);
21.        try {
22.            await access(requestedFile, constants.F_OK);
23.        } catch {
24.            throw new NotFoundException(`File ${filename} not found in stream folder.`);
25.        }
26.
27.        const fileStream = createReadStream(requestedFile);
28.
29.        fileStream.on('error', (error) => {
30.            this.logger.error(`Stream error for file ${requestedFile}:`, error);
31.        });
32.
33.        const contentType = filename.endsWith('.m3u8')
34.            ? 'application/vnd.apple.mpegurl'
35.            : 'video/mp2t';
36.
37.        return new StreamableFile(fileStream, { type: contentType });
38.    } catch (err) {}
39.
```

Kód 2 - streamVideo()

2.2.3 Ovládání přehrávání ve WatchParty

Tato metoda handleTimeUpdate se stará o automatické ukládání postupu sledování a chytré zobrazování návrhů na další díl, aby uživatel mohl plynule pokračovat tam, kde přestal.

V praxi to funguje tak, že kód neustále hlídá aktuální čas videa a porovnává ho s časem posledního uložení. Aby se zbytečně nepřetěžoval server, k zápisu do databáze dojde až ve chvíli, kdy se uživatel v příběhu posune o více než 5 sekund. V ten moment se zavolá funkce saveProgress, která na pozadí uloží přesnou vteřinu filmu nebo seriálu.

Zajímavou vychytávkou je proměnná isFinished. Kód počítá s tím, že na konci videa bývají titulky, takže video označí za „dokoukané“ už 30 sekund před koncem. Pokud jde o seriál, aplikace v tenhle moment automaticky připraví vyskakovací okno (overlay) s nabídkou další epizody, což uživateli šetří klikání. Celé je to navržené tak, aby systém sám poznal, jestli zrovna sledujete samostatný film nebo díl seriálu, a podle toho správně přiřadil ID k vašemu profilu. Je to v podstatě tichý pomocník, který hlídá, abyste nikdy neztratili nit a měli další díl hned po ruce.

```
1.      function handleTimeUpdate(detail: any) {
2.          const currentTime = detail.currentTime;
3.          const duration = player.current?.state.duration || 0;
4.
5.          if (Math.abs(currentTime - lastSavedTime.current) > 5) {
6.              lastSavedTime.current = currentTime;
7.              if (currentVideo && id) {
8.                  const isFinished = duration > 0 && currentTime > duration - 30;
9.
10.                 // Show overlay for series if near end
11.                 if (isFinished && currentVideo.videotype === 'SERIES' && !showNextOverlay)
{
12.                     const next = getNextEpisode();
13.                     if (next) setShowNextOverlay(true);
14.                 }
15.
16.                 saveProgress(user.token!, {
17.                     videoId: currentVideo.videotype === 'MOVIE' ? id : undefined,
18.                     episodeId: currentVideo.videotype === 'SERIES' ? id : undefined,
19.                     position: Math.floor(currentTime),
20.                     isFinished
21.                 });
22.             }
23.         }
24.     }
25.
```

Kód 3 - handlePlayback_action

2.2.4 Modal Provider

Komponenta ModalProvider nabízí vskutku zajímavý postup renderování modalů. Tato komponenta se sama nasazuje a funguje jako overlay který je permanentně kreslený přes celou stránku, ale nic nezobrazuje, pokud některý z modalu není vyvolán a nastaven jako že se má zobrazit. Pomocí zustandu je vytvořeno kontextové úložiště, které je základní struktura pro každý modal. Obsahuje základní parametry jako zda je otevřen, o jaký modal se jedná, metody pro otevření a zavření modalu. a také jaké modaly tento provider obsahuje.

```
1. export function ModalProvider() {
2.     const [isMounted, setIsMounted] = useState(false);
3.
4.     useEffect(() => {
5.         setIsMounted(true)
6.     }, [])
7.
8.     if(!isMounted) return null;
9.
10.    return(
11.        <>
12.            <VideoModal />
13.            <ProfileModal />
14.            <PartyModal />
15.            <JoinPartyModal />
16.        </>
17.    )
18. }
19.
```

Kód 4 - ModalProvider() + ModalStore

2.2.5 Dynamické přidávání nastavení

Tento kód využívá metodu Object.entries k iteraci přes konfigurační objekt settingsConfig, což umožňuje dynamické generování navigačních položek v rámci rozhraní nastavení. Každá položka seznamu (element <li>) je vybavena unikátním klíčem a obslužnou funkcí onClick, která po kliknutí aktualizuje stav activeTab na hodnotu odpovídající danému klíči, čímž zajišťuje přepínání mezi jednotlivými sekcemi. Vizuální stránka je řešena pomocí dynamických CSS tříd frameworku Tailwind; systém kontroluje, zda se aktuální klíč shoduje s aktivní záložkou, a na základě toho aplikuje buď zvýrazněný styl pro aktivní stav, nebo decentnější vzhled s efektem při najetí myší pro stavy neaktivní. Celý proces doplňuje vykreslení příslušné ikony a názvu sekce, což vede k vytvoření konzistentního a uživatelsky přívětivého bočního panelu, který lze snadno rozšiřovat o další možnosti nastavení pouhou úpravou zdrojového objektu.

```
 1.                {Object.entries(settingsConfig).map(([key, { name, icon: Icon }]) => (
 2.                  <li
 3.                    key={key}
 4.                    onClick={() => setActiveTab(key as keyof typeof settingsConfig)}
 5.                   className={`w-full flex items-center gap-4 px-4 py-3 text-lg cursor-pointer
rounded-md transition ${
 6.                      activeTab === key
 7.                        ? "bg-gray-700/40 text-white"
 8.                        : "text-gray-400 hover:bg-gray-600/20"
 9.                    }`}
10.                  >
11.                    <Icon className="w-5 h-5" />
12.                    <span>{name}</span>
13.                  </li>
14.                ))}
15.
```

Kód 5 - Funkce pro načtení nastavení

2.2.6 Zabezpečené Přihlašování

Tento kód řeší bezpečné ověření uživatele. Nejdříve se v databázi vyhledá uživatel podle e-mailu a následně se pomocí knihovny argon2 porovná zadané heslo s jeho zahashovanou podobou. Pokud vše sedí, server vygeneruje JWT token, který slouží jako digitální propustka pro další požadavky.

```
1. const user = await this.prisma.user.findUnique({ where: { email } });
2. if (!user) throw new UnauthorizedException('Neplatné údaje');
3.
4. const isPasswordValid = await argon2.verify(user.password, password);
5. if (!isPasswordValid) throw new UnauthorizedException('Neplatné údaje');
6.
7. const payload = { sub: user.id, email: user.email };
8. return { access_token: await this.jwtService.signAsync(payload) };
9.
```

Kód 5 - loginUser()

2.2.7 Synchronizace eventů WatchParty

Tato část kódu je srdcem funkce WatchParty. Využívá technologii Socket.io k tomu, aby jakákoliv akce (např. kliknutí na pauzu) okamžitě doputovala ke všem ostatním v místnosti. Místo aby se zpráva posílala každému zvlášť, využívám metodu to(roomId).emit, která zajistí hromadné rozeslání události všem připojeným klientům v reálném čase.

```
1. @SubscribeMessage('playback-action')
2. handlePlaybackAction(@ConnectedSocket() client: Socket, @MessageBody() data: any) {
3.   const roomId = this.roomService.getUserRoom(client.id);
4.   if (roomId) {
5.     // Rozeslání akce (play/pause/seek) všem členům v místnosti
6.     this.server.to(roomId).emit('remote-playback-action', data);
7.     this.logger.log(`Akce ${data.action} v místnosti ${roomId}`);
8.   }
9. }
10.
```

Kód 6 - playbackAction event

2.2.8 Globální správa stavu přehrávače

Pro efektivní ovládání přehrávače z různých částí aplikace (např. z modulu WatchParty) používám knihovnu Zustand. Tento fragment definuje globální úložiště, které si "pamatuje" referenci na aktuální přehrávač a umožňuje s ním manipulovat odkudkoliv, aniž by bylo nutné složitě předávat data mezi komponentami (tzv. prop drilling).

```
1. export const usePartyStore = create<PartyState>((set) => ({
2.   remote: null, // Reference na dálkové ovládání přehrávače
3.   setRemote: (remote) => set({ remote }),
4.   handleAction: (data) => {
5.     const { remote } = get();
6.     if (!remote) return;
7.     // Logika pro vykonání vzdálené akce na lokálním přehrávači
8.     if (data.action === 'PLAY') remote.play();
9.     if (data.action === 'PAUSE') remote.pause();
10.   }
11. }));
12.
```

Kód 7 - Zustand Party Store

2.3 Popis pro uživatele

1. Jak začít

Než se pustíte do sledování, musíte si vytvořit účet.

- **Registrace:** Na úvodní stránce zvolte registraci. Vyberte si přezdívku, zadejte svůj e-mail a zvolte si bezpečné heslo.
- **Přihlášení:** Jakmile máte účet hotový, přihlaste se pomocí svých údajů na přihlašovací stránce.

2. Výběr filmu nebo seriálu

Po přihlášení uvidíte přehlednou nabídku videí.

- **Hledání:** Videa můžete procházet podle žánrů (např. akční, komedie), abyste snadněji našli to, na co máte právě chuť.
- **Spuštění:** Klikněte na náhledový obrázek videa a otevře se vám přehrávač.

3. Společné sledování (WatchParty)

To nejdůležitější na PlayStorm je funkce **WatchParty**, díky které vidí všichni účastníci video úplně stejně a ve stejný čas.

- **Vytvoření skupiny:** Pokud chcete být tím, kdo vybírá a ovládá film, klikněte u přehrávače na tlačítko pro vytvoření party.
- **Pozvání přátel:** Aplikace vám vygeneruje kód místnosti. Tento kód pošlete svým přátelům.
- **Připojení k přátelům:** Pokud vás pozval někdo jiný, stačí v menu zadat kód, který vám poslal, a připojit se k jeho sledování.

4. Ovládání videa v partě

Když sledujete video v partě, ovládání je společné.

- **Synchronizace:** Když někdo video pozastaví, zastaví se všem. Když ho spustí, rozeběhne se všem.
- **Přetáčení:** Pokud se chcete o kousek vrátit nebo něco přeskočit, stačí použít tlačítka pro přetočení. Video se posune o 10 sekund všem účastníkům najednou.

5. Nastavení profilu

V nastavení si můžete upravit svůj účet podle sebe.

- **Úprava údajů:** Můžete si zde změnit heslo nebo nahrát vlastní profilový obrázek.
- **Odhlášení:** Pokud už aplikaci nechcete používat, najdete v menu tlačítko pro odhlášení.

Závěr

V rámci této ročníkové práce se mi podařilo úspěšně naplnit hlavní cíle, které jsem si vytyčil na samotném začátku v zadávacím listu. Podařilo se mi vyvinout funkční webovou aplikaci PlayStorm, která slouží jako platforma pro streamování multimediálního obsahu. Stěžejním bodem celého projektu byla implementace funkce WatchParty, která umožňuje synchronizované sledování videa v reálném čase pro více uživatelů najednou. Tento prvek, který byl od počátku definován jako hlavní přidaná hodnota aplikace, je plně funkční a umožňuje skupině lidí sdílet zážitek ze sledování bez ohledu na jejich fyzickou vzdálenost.

Při zpětném pohledu na původní očekávání mohu konstatovat, že aplikace splňuje všechny základní technické i uživatelské požadavky. Uživatelé mají k dispozici systém pro registraci, přihlášení a správu profilu, včetně možnosti změny hesla či profilového obrázku. Z hlediska backendu se mi podařilo vytvořit robustní API ve frameworku NestJS a propojit jej s databází PostgreSQL prostřednictvím ORM nástroje Prisma. Velkým úspěchem je také automatizovaný skript využívající nástroj FFmpeg, který samostatně zpracovává seriály a filmy, konvertuje je do formátu m3u8 playlistů a ukládá potřebné informace do databáze. Tento proces výrazně usnadňuje správu obsahu na media serveru.

Samozřejmě se během vývoje objevily i technické výzvy, které vyžadovaly úpravy původních plánů. Původní High-Level a Low-Level návrhy architektury musely být v průběhu produktizace částečně modifikovány, aby lépe odpovídaly reálným potřebám real-time komunikace přes Socket.io. Přestože byla synchronizace přehrávání (pauza, přetáčení, spuštění) nejkomplexnější částí projektu, výsledná logika v metodě handlePlaybackAction funguje spolehlivě.

Tento projekt mi přinesl obrovské množství nových zkušeností a hlubší pochopení systémového designu streamovacích platforem. Naučil jsem se efektivně pracovat s moderním technologickým stackem, zejména s Typescriptem a knihovnou Zustand pro správu globálních stavů aplikace. Velké poděkování patří mému vedoucímu práce, panu Bc. Tomáši Ulrichovi, za jeho odborné vedení, cenné rady při řešení technických problémů a pomoc s gramatickou kontrolou textu. Do budoucna plánuji aplikaci PlayStorm dále rozvíjet, přidat širší možnosti kategorizace a personalizace obsahu a případně ji nasadit pro širší okruh uživatelů jako komunitní platformu pro společné sledování anime a seriálů.

Seznam použitých zdrojů

1. **NETFLIX.** Netlix Logo. 2024.

2. **WATCH2GETHER.** Watch2Gether Logo. 2024.

3. **Vitejs.dev.** Vite. *Vite: Next Generation Frontend Tooling.* [Online] 2024. [Citace: 17. 1 2026.] https://vitejs.dev/.

4. **Meta Platforms.** React. *React: The library for web and native user interfaces.* [Online] 2024. [Citace: 17. 1 2026.] https://react.dev/.

5. **Microsoft.** TypeScript. *TypeScript: JavaScript with syntax for types.* [Online] 2024. [Citace: 17. 1 2026.] https://www.typescriptlang.org/.

6. **Tailwind Labs Inc.** Tailwind CSS. *Tailwind CSS: Rapidly build modern websites...* [Online] 2024. [Citace: 17. 1 2026.] https://tailwindcss.com/.

7. **Socket.io.** Socket.IO. *Socket.IO: Bidirectional and low-latency...* [Online] 2024. [Citace: 17. 1 2026.] https://socket.io/.

8. **Vidstack.** Vidstack. *Vidstack: High-performance media player...* [Online] 2024. [Citace: 17. 1 2026.] https://www.vidstack.io/.

9. **GitHub, Inc.** Zustand. *Zustand: A small, fast and scalable bearbones...* [Online] 2024. [Citace: 17. 1 2026.] https://github.com/pmndrs/zustand.

10. **Nestjs.com.** NestJS. *NestJS: A progressive Node.js framework...* [Online] 2024. [Citace: 17. 1 2026.] https://nestjs.com/.

11. **Prisma Data, Inc.** Prisma. *Prisma: Next-generation Node.js and TypeScript ORM.* [Online] 2024. [Citace: 17. 1 2026.] https://www.prisma.io/.

12. **The PostgreSQL Global Development Group.** PostgreSQL. *PostgreSQL: The World's Most Advanced...* [Online] 2024. [Citace: 17. 1 2026.] https://www.postgresql.org/.

13. **Auth0.** JWT. *JSON Web Tokens: Introduction.* [Online] 2024. [Citace: 17. 1 2026.] https://jwt.io/introduction/.

14. **Ffmpeg.org.** FFmpeg. *FFmpeg: A complete, cross-platform solution...* [Online] 2024. [Citace: 17. 1 2026.] https://ffmpeg.org/.

15. **Python Software Foundation.** Python. *Python Programming Language.* [Online] 2024. [Citace: 17. 1 2026.] https://www.python.org/.

16. **Docker Inc.** Docker. *Docker: Accelerated Container Application...* [Online] 2024. [Citace: 17. 1 2026.] https://www.docker.com/.

Seznam obrázků

Obrázek 1 - Netflix Logo 9
Obrázek 2 - Watch2Gether Logo 9
Obrázek 3 - ERD Diagram 12
Obrázek 4 - Navrh Backendu a API 13

Seznam kódů

Kód 1 - getVideo() 14
Kód 2 - streamVideo() 15
Kód 3 - handlePlayback_action 16
Kód 4 - ModalProvider() + ModalStore 17
Kód 5 - loginUser() 19
Kód 6 - playbackAction event 20
Kód 7 - Zustand Party Store 21